Amino Farms feed mill: two additions to the SCADA project

Plant: Amino Farms feed mill, Dhekiajuli. **SCADA PC:** `DESKTOP-CK6AQJR`. **Project:** TIA Portal V15.1, `D:\DO NOT DELETE\AMINOFARMS\AMINOFARMS`, HMI device **SCADA [WinCC RT Professional]** (runtime project `D:\HMI_5TG9`). **SQL Server:** `.\WINCC`.

Amino Farms runs its own system, **niko** (aminofarms.com). A small helper program on the SCADA PC already reads `BATCH.dbo.HISTORY` and sends each batch to niko. That is working today. We now need two additions inside the WinCC project.

	Request	Direction	Changes plant behaviour?
A	Live values for a read-only "Live Mill" web screen	SCADA → niko	No
B	Import a recipe prepared in niko	niko → SCADA	Only when the operator imports and loads it

Rules niko keeps in both:

- niko never talks to the PLC and never writes a WinCC tag.
- niko never writes the recipe archive `UA#BATCHING` directly. WinCC writes its own archive.
- The operator still loads and starts every batch at the SCADA, as today.
- No OPC server and no new network port are needed.

Both can be done in one visit. Both need a compile and download **with the mill stopped**.

A. Live values

What we need

One VB script that writes the current value of **133 tags** (listed in the appendix) into **one SQL table** every **2 seconds**. 5 seconds is acceptable. It works the same way as your existing batch logging (`ScriptAct\datalog.bac`).

A1. The table (create once, in the existing `BATCH` database)

```
USE BATCH;
CREATE TABLE dbo.NIKO_LIVE (
    TagName   nvarchar(100) NOT NULL PRIMARY KEY,
    TagValue  nvarchar(200) NULL,
    UpdatedAt datetime      NOT NULL DEFAULT GETDATE()
);
```

Column	Type	Meaning
`TagName`	nvarchar(100), key	The WinCC tag name, exactly as in the appendix
`TagValue`	nvarchar(200), null	The current value, as text
`UpdatedAt`	datetime	When the script last wrote this row (PC time)

One row per tag, overwritten in place, so the table never grows.

A2. The script

- **Trigger:** cyclic, every 2 s (5 s if 2 s is not possible).
- **Each run:** read the 133 tags and write all of them in one statement or transaction: update the row if the tag exists, insert it if not.
- **`UpdatedAt`:** set to `GETDATE()` on every row, every run, even when the value did not change. niko uses it to know the values are current.
- **Values:** as text, as WinCC shows them. Booleans as `1`/`0` or `True`/`False`. Text tags (bin names, recipe name) unchanged.
- **A tag that cannot be read:** write `NULL` and carry on with the rest.
- **Connection:** the same as `datalog.bac`. Read access for the Windows user `amino` is enough for niko's side.

Outline, to adapt to your conventions:

```
Dim tags, conn, sql, i, v
tags = Array("BATCHINGDATA1_LOADCELL", "BATCHINGDATA1_LOADCELL-2")  ' ... all 133
Set conn = CreateObject("ADODB.Connection")
conn.Open "Provider=SQLOLEDB;Data Source=.\WINCC;Initial Catalog=BATCH;Integrated Security=SSP
I;"
sql = "SET NOCOUNT ON; BEGIN TRAN;"
For i = 0 To UBound(tags)
    v = HMIRuntime.Tags(tags(i)).Read
    sql = sql & "MERGE dbo.NIKO_LIVE AS t USING (SELECT N'" & tags(i) & "' AS n) AS s ON t.TagN
ame = s.n " & _
          "WHEN MATCHED THEN UPDATE SET TagValue = N'" & Replace(CStr(v), "'", "''") & "', Upda
tedAt = GETDATE() " & _
          "WHEN NOT MATCHED THEN INSERT (TagName, TagValue) VALUES (s.n, N'" & Replace(CStr(v),
"'", "''") & "');"
Next
sql = sql & "COMMIT;"
conn.Execute sql
conn.Close
```

A3. Check

This should return 133 rows, with `UpdatedAt` in the last few seconds and values matching the batching screen:

```
SELECT TagName, TagValue, UpdatedAt FROM BATCH.dbo.NIKO_LIVE ORDER BY TagName;
```

B. Recipe import from niko

What we need

An **"Import from niko"** button on the recipe screen. It reads one recipe that niko has prepared and writes it into the `BATCHING` user archive using WinCC's own user-archive functions, so the archive's fingerprint stays valid.

B1. The archive, as we see it today (read-only survey, 6 Oct 2026)

`CC_HMI_5TG9_26_09_03_12_34_53R.dbo.UA#BATCHING`

Column	Meaning
`ID`	Record number
`Name`	Recipe name (today: Layer 1, Layer 2, Layer 3, Pre Layer)
`BIN_1` … `BIN_8`	Set weight per bin, kg
`LastUser`, `LastAccess`	Written by WinCC
`Fingerprint`	WinCC's integrity check: the reason niko will not write this table

The related tags are `BATCHINGDATA1_RECEIPESETWG{1}` … `{6}`, `BATCHINGDATA1_RECEIPEC-SETWG{7}`, `{8}` and `COMMONDB_RECEIPE_NAME`.

B2. What niko will deliver: one file per recipe

The helper on the SCADA PC writes a small text file into a folder you choose. Our proposal is `D:\HMI_5TG9\niko_recipes\`, one file per recipe, named after the recipe, for example `Layer 3.csv`. A new push overwrites the file.

Proposed format: plain text, one `key,value` per line, UTF-8 without BOM.

```
Name,Layer 3
BIN_1,45
BIN_2,120
BIN_3,400
BIN_4,300
BIN_5,110
BIN_6,280
BIN_7,170
BIN_8,50
BINNAME_1,STONE
BINNAME_2,DORB
BINNAME_3,MAIZE
BINNAME_4,MAIZE
BINNAME_5,DDGS
BINNAME_6,SOYA
BINNAME_7,MAIZE
BINNAME_8,GNDOC
TOTAL,1475
```

```
PREPARED_AT,2026-10-14 10:30:00
NIKO_REF,Layer 3 v7
```

Line	Meaning	What the import does with it
`Name`	Recipe name	The archive record to update, or to create if it is new
`BIN_1` … `BIN_8`	Set weight per bin, kg	Written to the record
`BINNAME_1` … `8`	The material niko believes is in each bin	**Check only:** compare with the current bin names (`COMMONDB_BINNAME{1..8}`) and refuse the import if any differ
`TOTAL`	Sum of the eight bins	**Check only:** refuse if it does not equal the sum
`PREPARED_AT`, `NIKO_REF`	When and from which niko formula	Show to the operator; no need to store

The numbers above are an example, not a real recipe. If a different format is easier for you (fixed columns, XML, or a SQL request table in `BATCH`), tell us and we will produce that instead.

B3. What the button should do

1. Let the operator choose a file from the folder (or list the recipes found).
2. Show the recipe: name, kg per bin, total, when it was prepared.
3. Run the two checks in B2. On a mismatch, show which bin and import nothing.
4. On the operator's confirmation, write the record through the WinCC user-archive functions (update if the name exists, otherwise insert).
5. **Do not load it into the PLC.** The operator selects and loads the recipe as today.
6. Refuse to import over the recipe of a batch that is running, if the PLC reads the archive live (see question 1).

B4. Read-back

No work for you. The helper reads `UA#BATCHING` (SELECT only) after the import and niko shows matched or not matched for every bin.

Questions we need answered

1. **How does the PLC take a recipe?** Only when the operator loads it, or does it read the archive live while a batch runs?
2. **Limits per bin:** minimum and maximum kg for each of the 8 bins, and the fine-weigh (coarse/fine gate) settings we must respect.
3. **Weights:** whole kg only, or are decimals accepted in `BIN_1` … `BIN_8`?
4. **Batch size:** is there a fixed batch size or maximum total (our history shows about 1,475 kg)?
5. **One material in several bins** (maize is in three): does the SCADA need anything special, or are they simply three independent set weights?
6. **Recipe names:** any limit on length or characters? Is there a maximum number of recipes in the archive?
7. **Folder and format for B2:** is our proposal acceptable?
8. **Time needed** on site, and how long the mill must be stopped.

Appendix: the 133 tags for request A

All of these are already in the project (connection `HMI_Connection_1`).

```
BATCHINGDATA1_ACTUALBINWG{1}
BATCHINGDATA1_ACTUALBINWG{2}
BATCHINGDATA1_ACTUALBINWG{3}
BATCHINGDATA1_ACTUALBINWG{4}
BATCHINGDATA1_ACTUALBINWG{5}
BATCHINGDATA1_ACTUALBINWG{6}
BATCHINGDATA1_ACTUALBINWG{7}
BATCHINGDATA1_ACTUALBINWG{8}
BATCHINGDATA1_BATCHINGSTEP1
BATCHINGDATA1_BATCHSTART
BATCHINGDATA1_BINC-GATEOUT{1}
BATCHINGDATA1_BINC-GATEOUT{2}
BATCHINGDATA1_BINC-GATEOUT{3}
BATCHINGDATA1_BINC-GATEOUT{4}
BATCHINGDATA1_BINC-GATEOUT{5}
BATCHINGDATA1_BINC-GATEOUT{6}
BATCHINGDATA1_BINC-GATEOUT{7}
BATCHINGDATA1_BINC-GATEOUT{8}
BATCHINGDATA1_BINF-GATEOUT{1}
BATCHINGDATA1_BINF-GATEOUT{2}
BATCHINGDATA1_BINF-GATEOUT{3}
BATCHINGDATA1_BINF-GATEOUT{4}
BATCHINGDATA1_BINF-GATEOUT{5}
BATCHINGDATA1_BINF-GATEOUT{6}
BATCHINGDATA1_BINF-GATEOUT{7}
BATCHINGDATA1_BINF-GATEOUT{8}
BATCHINGDATA1_COMPLETEBATCH
BATCHINGDATA1_CURRENTBATCH
BATCHINGDATA1_CURRENTBIN
BATCHINGDATA1_CURRENTBIN-2
BATCHINGDATA1_HOLD
BATCHINGDATA1_INFLIGHTVALUE{1}
BATCHINGDATA1_INFLIGHTVALUE{2}
BATCHINGDATA1_INFLIGHTVALUE{3}
BATCHINGDATA1_INFLIGHTVALUE{4}
BATCHINGDATA1_INFLIGHTVALUE{5}
BATCHINGDATA1_INFLIGHTVALUE{6}
BATCHINGDATA1_INFLIGHTVALUE{7}
BATCHINGDATA1_INFLIGHTVALUE{8}
BATCHINGDATA1_LOADCELL
BATCHINGDATA1_LOADCELL-2
BATCHINGDATA1_RECEIPEC-SETWG{7}
BATCHINGDATA1_RECEIPEC-SETWG{8}
BATCHINGDATA1_RECEIPESETWG{1}
BATCHINGDATA1_RECEIPESETWG{2}
BATCHINGDATA1_RECEIPESETWG{3}
BATCHINGDATA1_RECEIPESETWG{4}
BATCHINGDATA1_RECEIPESETWG{5}
BATCHINGDATA1_RECEIPESETWG{6}
```

```
BATCHINGDATA1_SETBATCHCOUNT
BATCHINGDATA1_SETMIXINGTMR{1}
BATCHINGDATA1_SETMIXINGTMR{2}
BATCHINGDATA1_TOTALACT
BATCHINGDATA1_TOTALSET
BBCON_ANIMATION
COMMONDB_AUTOCMD
COMMONDB_BINNAME{1}
COMMONDB_BINNAME{2}
COMMONDB_BINNAME{3}
COMMONDB_BINNAME{4}
COMMONDB_BINNAME{5}
COMMONDB_BINNAME{6}
COMMONDB_BINNAME{7}
COMMONDB_BINNAME{8}
COMMONDB_BUZZER
COMMONDB_EMERGENCYSTOP
COMMONDB_GINDER-1SP
COMMONDB_RECEIPE_NAME
COMMONDB_SILONAME{1}
COMMONDB_SILONAME{2}
COMMONDB_SILONAME{3}
COMMONDB_SILONAME{4}
COMMONDB_SILONAME{5}
DISTRIBUSION_CHAIN_CONV-2_ANIMATION
DISTRIBUSION_CHAIN_CONV-3_ANIMATION
DISTRIBUSION_CHAIN_CONV_ANIMATION
FFD_SCREW_CONVYOR_ANIMATION
FF_ELEVATOR-1_ANIMATION
FF_TOP_CONVYOR-1_ANIMATION
GRINDERELEVATOR_ANIMATION
GRP_1DATA_BINHIGHSENSOR{1}
GRP_1DATA_BINHIGHSENSOR{2}
GRP_1DATA_BINHIGHSENSOR{3}
GRP_1DATA_BINHIGHSENSOR{4}
GRP_1DATA_BINHIGHSENSOR{5}
GRP_1DATA_BINHIGHSENSOR{6}
GRP_1DATA_BINHIGHSENSOR{7}
GRP_1DATA_BINHIGHSENSOR{8}
GRP_1DATA_BINSCADAGATESEL{1}
GRP_1DATA_BINSCADAGATESEL{2}
GRP_1DATA_BINSCADAGATESEL{3}
GRP_1DATA_BINSCADAGATESEL{4}
GRP_1DATA_BINSCADAGATESEL{5}
GRP_1DATA_BINSCADAGATESEL{6}
GRP_1DATA_BINSCADAGATESEL{7}
GRP_1DATA_BINSCADAGATESEL{8}
GRP_1DATA_DUMPLAMP-1
GRP_1DATA_DUMPLAMP-2
GRP_1DATA_FLAPGATE
GRP_1DATA_GP_1SCADAONCMD
GRP_2DATA_GP_2SCADAONCMD
GRP_2DATA_GRINDERHOLD
GRP_3DATA_FFFLAPGATE
```

```
GRP_3DATA_GP_3SCADAONCMD
GRP_3DATA_GRP3GATES{1}
GRP_3DATA_GRP3GATES{2}
GRP_3DATA_GRP3GATES{3}
GRP_3DATA_GRP3GATES{4}
GRP_3DATA_GRP3GATES{5}
GRP_3DATA_GRP3SENSORS{1}
GRP_3DATA_GRP3SENSORS{2}
GRP_3DATA_GRP3SENSORS{3}
GRP_3DATA_GRP3SENSORS{4}
GRP_3DATA_GRP3SENSORS{5}
GRP_3DATA_HOLD
MIXELEVATORSCCON_ANIMATION
MIXELEVATOR_ANIMATION
MIXERAMPS_realout
MIXER_ANIMATION
RM_ELEVATOR-2_ANIMATION
RM_ELEVATOR_ANIMATION
RM_MOTOR-1_ANIMATION
ROTARYHZ_realout
RWMATRLLLFB-1
RWMATRLLLFB-2
RWMATRLLLFB-3
RWMATRLLLFB-4
RWMATRLLLFB-5
RWMATRLLLFB-6
RWMATRLLLFB-7
RWMATRLLLFB-8
grinderamps-2_realout
grinderamps_realout
```